Phase 1 - Grade 5 - Jupyter Notebook Workflow

Key Learning Points

A Jupyter Notebook is a document made of **cells** -- small chunks of code (or text) run one at a time, independently, with output appearing directly below and staying there. Real ML/data work is a 'load data -> look -> tweak -> look again' loop; notebooks let you load data ONCE, then keep experimenting without re-running that expensive step.

Analogy that made it click: like a Java debugger session where you inspect variables at a breakpoint -- except each cell IS a persistent, independently re-runnable checkpoint, and the 'variables' stay alive in memory across cells until the whole notebook is restarted.

Kernel
: The single running Python process behind a notebook. ALL cells share ONE kernel's memory, regardless of position on the page.

Cell types
: Code cells run actual Python. Markdown cells hold formatted text/notes only -- no code executes there.

Execution order vs. page order
: Cells share memory based on the order they were ACTUALLY RUN, not their position on the page. Running cells out of sequence is the #1 source of beginner notebook bugs.

In [n]
: Shows the ORDER a cell was executed in (1st, 2nd, 3rd overall run), NOT its page position, and NOT a guarantee the cell's current visible code still matches what actually produced that output (if edited afterward without re-running).

Restart Kernel + Run All
: Wipes all memory clean, then runs every cell top-to-bottom in page order. The ONLY action that fully guarantees the notebook's visible code and output truly correspond -- mandatory before trusting or sharing results.

Examples

Worked example -- stale state: Cell A: y = 5. Cell B (run after A): print(y * 2) -> shows 10. Cell A is then EDITED to y = 50 but NOT re-run. Cell B is re-run alone: it STILL prints 10, not 100 -- the kernel's memory still holds the OLD value; the edit to Cell A never actually executed. The notebook's displayed text and its live memory are out of sync until Cell A is re-run.

Quiz Q&A;

Q1. A notebook shows In [1], In [2], In [3] in exact page order. Does this GUARANTEE trustworthy/reproducible results?

A: No. Sequential, page-matching numbering only shows execution order matched page order AT THE TIME each cell was last run -- it does not prove a cell wasn't edited afterward without re-running. Only Restart Kernel + Run All gives a real guarantee.

Q2. You get NameError: name 'df' is not defined even though a cell above (on the page) clearly defines df = pd.DataFrame(...). Two possible explanations?

A: (1) That defining cell was simply never actually run. (2) The kernel was restarted or crashed since it last ran -- wiping all memory even though the page still shows the old output from before the crash.

Q3. True or False: 'Variables defined in one cell are isolated to that cell only, like local variables in a function.'

A: False. Variables are shared across the ENTIRE notebook once the defining cell has successfully run.

Q4. What keyboard shortcut runs the current cell and moves to the next one?

A: Shift + Enter.

Q5. Why is a notebook a better fit than a plain .py script for 'load data once, explore repeatedly'? Why might a script be better for production?

A: A notebook keeps data loaded in live memory across cells, avoiding an expensive re-load every time. A plain script is better for production because it runs reliably, deterministically, top-to-bottom, with no possibility of hidden/stale state from out-of-order execution.